

A Beginner-Friendly Overview of Neural Networks

Introduction (Hook):

Can a neural network learn what makes a video game highly rated? That was the challenge our team took on. Using a dataset of over 47,000 games, we trained a neural network in PyTorch to predict user ratings based on features like price, genre, publisher, and game length.

Along the way, we explored key concepts in deep learning, from embeddings to backpropagation, and discovered just how powerful neural networks can be when applied to structured datasets.

Before we dive deeper, you might be wondering: how exactly do neural networks work?

At their core, neural networks are designed to mimic the way the human brain processes information. They take in input data such as an image, a sentence, or a spreadsheet row and pass it through multiple layers of interconnected nodes, also called neurons. Each connection between neurons has a weight that determines its influence. As the data flows through the network, these weights are combined with a bias (a small adjustment), and activation functions determine which signals are strong enough to pass forward.

The first layer captures basic information, while hidden layers detect patterns and features. The final output layer produces a prediction in our case, a user rating. Learning occurs through a process called gradient descent, where the network makes a guess, measures how wrong it was (the loss), and slightly adjusts its weights and biases to improve the next prediction. Over many epochs (repeated passes through the training data), the model becomes better at recognizing patterns and making accurate predictions without ever being explicitly told the rules. As demonstrated by the visual below.

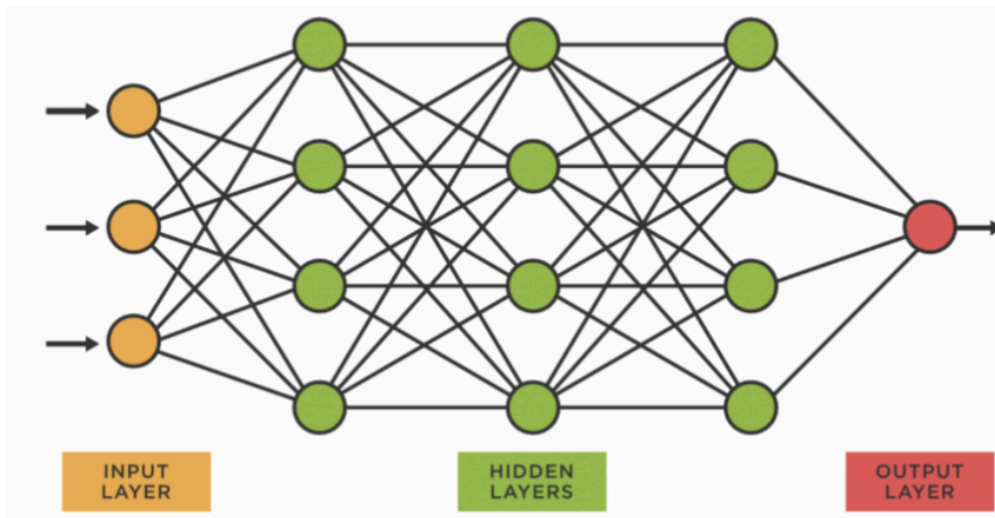


Image Credit: <https://www.spotfire.com/glossary/what-is-a-neural-network>

The Tabular Methods:

Our dataset included over 47,000 video games and 18 columns, combining both numerical and categorical features. Numerical fields like price, release year, game length, and minimum number of players offered measurable data points. Categorical features ranged from binary indicators like multiplayer support to high-cardinality fields such as publisher, genre, platform, and developer, some with hundreds of unique values. As shown down below.

	Game Title	User Rating	Age Group Targeted	Price	Platform	Requires Special Device	Developer	Publisher	Release Year	Genre	Multiplayer	Game Length (Hours)	Graphics Quality	Soundtrack Quality	Story Quality	User Review Text	Game Mode	Min Number of Players
0	Grand Theft Auto V	36.4	All Ages	41.41	PC	No	Game Freak	Innersloth	2015	Adventure	No	55.3	Medium	Average	Poor	Solid game, but too many bugs.	Offline	1
1	The Sims 4	38.3	Adults	57.56	PC	No	Nintendo	Electronic Arts	2015	Shooter	Yes	34.6	Low	Poor	Poor	Solid game, but too many bugs.	Offline	3
2	Minecraft	26.8	Teens	44.93	PC	Yes	Bungie	Capcom	2012	Adventure	Yes	13.9	Low	Good	Average	Great game, but the graphics could be better.	Offline	5

To streamline the model, we removed non-essential columns like game titles and raw review text, which contained unstructured or inconsistent data that wouldn't contribute meaningfully to the prediction task. Instead, we focused on features most relevant to gameplay experience, such as price, game length, visual and audio quality, story, and release year. These features provided a balanced mix of quantitative and categorical information, giving the model a clearer foundation to learn from.

A key challenge with structured data is helping the model interpret both numbers and categories effectively. Neural networks can't process text labels directly, so we grouped features by type and applied embedding layers to our categorical columns. For example, instead of assigning a publisher like "Nintendo" a simple numeric index, an embedding creates a learnable vector that captures deeper relationships — such as how certain publishers consistently release higher-rated games.

Embedding:

Embeddings reduce the complexity of our input space while preserving important patterns across categories. This allowed the model to recognize that games from similar genres, platforms, or developers often share rating trends, even if they are not identical. Instead of treating each category as completely separate, embeddings place them in a continuous vector space where relationships can be learned and refined during training. For example, if two platforms tend to host similar types of games or attract similar audiences, their embeddings will naturally be positioned closer together.

Compared to traditional encoding methods such as one-hot encoding, embeddings are more compact, scalable, and efficient—especially when dealing with high-cardinality data like publishers or genres with hundreds of unique values. One-hot encoding treats every category as unrelated, increasing input size and ignoring potential connections between values. Embeddings, on the other hand, enable the model to learn those relationships as part of the prediction process.

Although neural networks were originally designed for unstructured data like images and text, most real-world information—particularly in areas such as gaming, healthcare, and finance—is organized in rows and columns. This structured format presents a challenge because models must learn to interpret both numerical and categorical inputs in meaningful ways. Traditional neural networks often struggled with this, which led to the development of specialized tabular methods that adapt neural architectures to structured data. These innovations, including embeddings and feature engineering techniques, allowed our model to uncover deeper patterns and improve prediction accuracy throughout the project.

PyTorch:

We implemented our model using PyTorch, a deep learning framework that gave us the flexibility to define, train, and fine-tune our neural network from start to finish. To evaluate performance fairly, we split our dataset into 80 percent training and 20 percent testing, ensuring the model had enough data to learn from while still allowing us to assess how well it generalized to new examples.

Training took place over 30 epochs, with the model gradually improving its predictions through gradient descent. Gradient descent essentially tells the model how far off it is and in what direction it should adjust its internal weights to get closer to the correct answer. Each adjustment is called a 'step,' and the model takes many small steps over time—across epochs—which allows it to gradually lower the loss and improve its predictions.

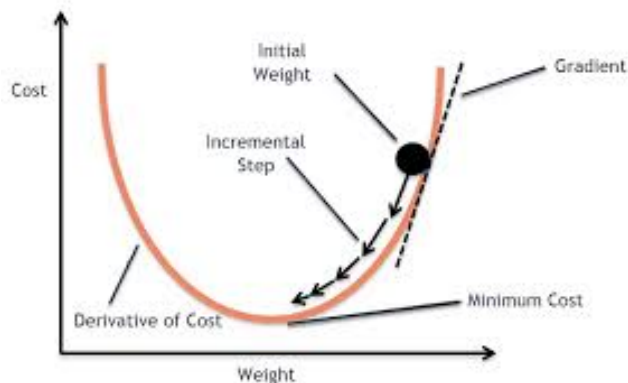


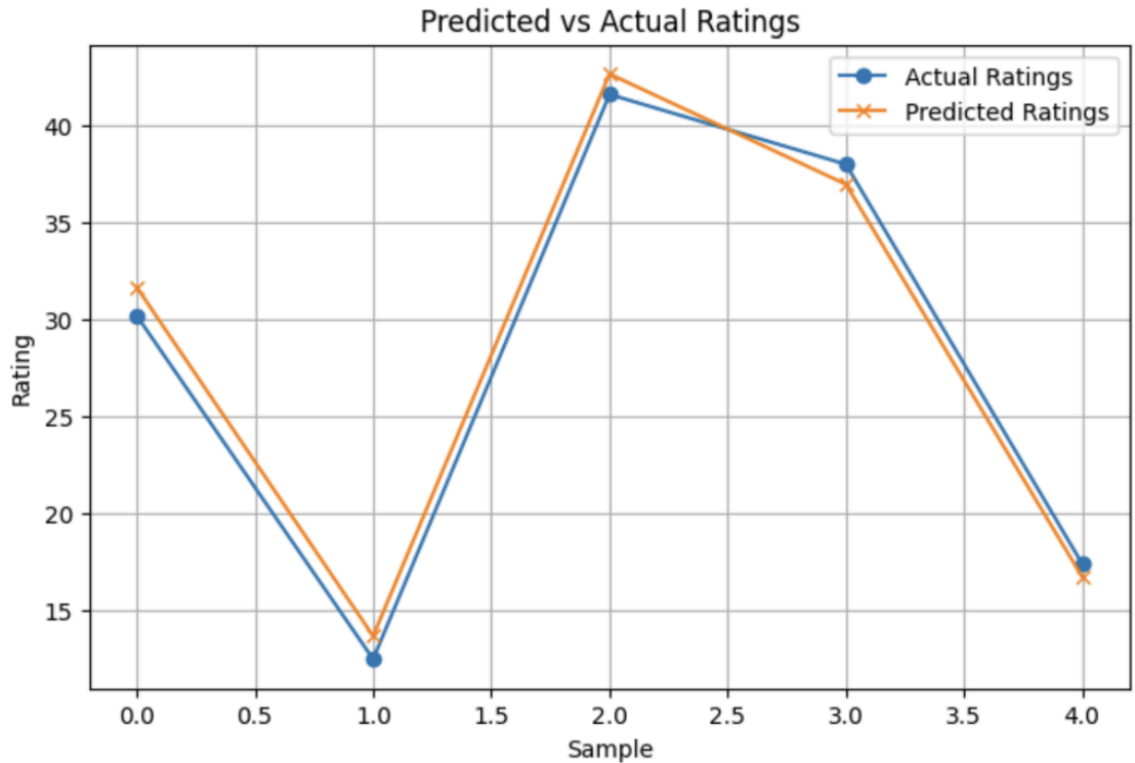
Image Credit:

<https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>

As you can imagine our initial model was not accurate, but, through gradient descent it recognized it was wrong, and took a step in the right direction (in the photo that would be to the left)

To guide this process, we used Root Mean Square Error (RMSE) as our loss function, a crucial tool for measuring how far off each prediction was from the actual user rating. RMSE is especially useful in regression tasks like ours because it penalizes larger errors more strongly. This pushes the model not only to be generally accurate, but also to avoid big mistakes. In the early epochs, RMSE was over 28, indicating high initial error. But as training progressed, the model adjusted its internal weights to reduce this loss, ultimately reaching an RMSE of around 1.2 on both the training and validation sets.

This significant drop confirmed that the network was learning meaningful patterns in the data, not just memorizing, and could generalize its predictions to new, unseen games.



As we can see here our model is very close to the actual ratings, it follows the same trends of dropping and rising depending on the rating itself. Now it is important to note that this isn't perfect but it's close which can be explained by the small gaps that are in between the actual and predicted trends demonstrating that there are imperfections between the model.

Conclusion:

This project gave us the opportunity to take deep learning out of the abstract and apply it to a real-world challenge. By combining structured data, embedding techniques, and the flexibility of PyTorch, we built a model that could make accurate, interpretable predictions about video game user ratings.

More importantly, we learned how data preparation, architecture design, and evaluation metrics all work together to shape model performance. Projects like this show how machine learning can extract meaningful insights from complex datasets — not just in gaming, but in any industry where data can help drive smarter decisions. Below will be a link to the GitHub Profile where you'll be able to see exactly our steps and the approach we took within the process of constructing the model itself.

Thank you for reading!